

Kivy Basics — A Beginner's Guide

1. What is Kivy?

Kivy is an open-source Python framework for building cross-platform applications with natural user interfaces. With one Kivy codebase, you can build an app that runs on:

- Windows, macOS, Linux (desktop)
- Android and iOS (mobile)
- Raspberry Pi

Kivy is popular for apps that need custom UI, touch/gesture support, or multi-touch input (drawing apps, games, kiosks, IoT dashboards), though it's used for plenty of ordinary business apps too.

Why people choose Kivy:

- Pure Python — no need to learn a separate native language for each platform
 - Built-in widget toolkit (buttons, labels, sliders, etc.)
 - GPU-accelerated rendering (uses OpenGL under the hood), so animations are smooth
 - Its own lightweight design language called **KV language** for separating UI from logic (similar in spirit to how HTML separates structure from JavaScript logic)
-

2. Installing Kivy

Kivy is installed like any other Python package, using pip:

```
python -m pip install --upgrade pip setuptools virtualenv
python -m venv kivy_venv
source kivy_venv/bin/activate    # On Windows: kivy_venv\Scripts\activate
python -m pip install kivy[base]
```

Using a **virtual environment** (venv) is strongly recommended so Kivy's dependencies don't clash with other projects on your machine.

Verify the install:

```
python -c "import kivy; print(kivy.__version__)"
```

3. The Anatomy of a Kivy App

Every Kivy application follows the same basic pattern:

1. You create a class that inherits from `kivy.app.App`.
2. You override the `build()` method — this returns the root widget (the top of your UI tree).
3. You call `.run()` to start the app's event loop.

Minimal Example

```
from kivy.app import App
from kivy.uix.label import Label

class MyFirstApp(App):
    def build(self):
        return Label(text="Hello, Kivy!")

if __name__ == "__main__":
    MyFirstApp().run()
```

Run it with:

```
python main.py
```

You should see a window with the text "Hello, Kivy!" centered on screen.

Key idea: The class name matters if you're pairing it with a `.kv` file (explained in Section 6) — Kivy looks for a `.kv` file whose name matches your App class, minus the word "App" (e.g., `MyFirstApp` → `myfirst.kv`).

4. Widgets — The Building Blocks

Widgets are the visual/interactive elements of your app. Kivy ships with many built-in ones:

Widget	Purpose
Label	Displays text
Button	Clickable button
TextInput	Text entry field
Image	Displays an image
Slider	Draggable value selector
CheckBox	Toggle on/off
Switch	iOS-style toggle
ProgressBar	Shows progress
Spinner	Dropdown-style selector
ScrollView	Scrollable container

Example – a clickable button that changes its own text:

```
from kivy.app import App
from kivy.uix.button import Button

class ButtonApp(App):
    def build(self):
        button = Button(text="Click Me")
        button.bind(on_press=self.on_button_click)
        return button

    def on_button_click(self, instance):
        instance.text = "Clicked!"

if __name__ == "__main__":
    ButtonApp().run()
```

Notice `button.bind(on_press=...)` – this is how Kivy handles **events**. You "bind" a function to an event, and Kivy calls that function whenever the event fires.

5. Layouts – Arranging Widgets

A single widget alone isn't a real app. **Layouts** are container widgets that arrange their children automatically. The most common ones:

- **BoxLayout** — arranges children in a row or column
- **GridLayout** — arranges children in a grid of rows/columns
- **FloatLayout** — positions children using relative (0–1) or absolute coordinates
- **AnchorLayout** — anchors children to an edge or corner
- **RelativeLayout** — like FloatLayout but positions are relative to the layout itself

Example: BoxLayout

```
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.button import Button
from kivy.uix.label import Label

class LayoutApp(App):
    def build(self):
        layout = BoxLayout(orientation='vertical', padding=10, spacing=10)
        layout.add_widget(Label(text="Enter your name:"))
        layout.add_widget(Button(text="Submit"))
        return layout

if __name__ == "__main__":
    LayoutApp().run()
```

orientation='vertical' stacks widgets top to bottom; 'horizontal' places them side by side. padding adds space around the edges, and spacing adds space between children.

6. The KV Language

Instead of building your entire UI in Python, Kivy lets you describe it in a separate, more readable file using **KV language** (.kv files). This keeps your UI (design) separate from your app logic (Python) — similar to how you'd separate HTML/CSS from JavaScript.

Example

main.py

```
from kivy.app import App

class GreetApp(App):
    def on_greet_press(self):
        self.root.ids.greet_label.text = "Hello there!"

if __name__ == "__main__":
    GreetApp().run()
```

greet.kv (must be named after the App class, lowercase, minus "App")

BoxLayout:

orientation: 'vertical'

Label:

id: greet_label

text: "Press the button below"

Button:

text: "Greet Me"

on_press: app.on_greet_press()

Kivy automatically loads greet.kv because the App class is named GreetApp. Inside the .kv file:

- Indentation defines parent-child widget relationships (like Python).
- id: lets you reference a widget elsewhere (via self.ids or app.root.ids).
- on_press: app.on_greet_press() calls a method on the running App instance.

This separation makes larger apps far easier to maintain.

7. Properties — Kivy's Reactive Variables

Kivy widgets use special variables called **Properties** (e.g., StringProperty, NumericProperty, BooleanProperty, ListProperty, ObjectProperty). Unlike plain Python attributes, Properties automatically notify listeners when their value changes — this is what makes KV bindings "reactive."

```
from kivy.app import App
from kivy.uix.label import Label
from kivy.properties import StringProperty

class ReactiveLabel(Label):
    display_text = StringProperty("Waiting...")

class ReactiveApp(App):
    def build(self):
        label = ReactiveLabel()
        label.display_text = "Updated automatically!"
        return label

if __name__ == "__main__":
    ReactiveApp().run()
```

In a .kv file, you could bind another widget's property directly to `display_text`, and it would update live without any manual event-handling code — this is one of Kivy's most powerful features.

8. Handling Events

Two common ways to respond to user interaction:

1. `bind()` in Python: `python button.bind(on_press=my_function)`
2. `on_<event>`: in KV: `kv Button: on_press: print("Button pressed!")`

Common built-in events: `on_press`, `on_release` (buttons), `on_text` (TextInput), `on_touch_down`, `on_touch_move`, `on_touch_up` (low-level touch handling).

9. Screen Management (Multi-Screen Apps)

Most real apps have more than one screen. Kivy provides `ScreenManager` for this:

```
from kivy.app import App
```

```

from kivy.uix.screenmanager import ScreenManager, Screen
from kivy.uix.button import Button

class HomeScreen(Screen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.add_widget(Button(text="Go to Settings", on_press=self.switch))

    def switch(self, instance):
        self.manager.current = 'settings'

class SettingsScreen(Screen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.add_widget(Button(text="Back to Home", on_press=self.switch))

    def switch(self, instance):
        self.manager.current = 'home'

class ScreenApp(App):
    def build(self):
        sm = ScreenManager()
        sm.add_widget(HomeScreen(name='home'))
        sm.add_widget(SettingsScreen(name='settings'))
        return sm

if __name__ == "__main__":
    ScreenApp().run()

sm.current = 'settings' switches the visible screen, with a built-in transition animation.

```

10. Project Structure (Typical Layout)

```

my_app/
├── main.py      # App entry point
├── myapp.kv     # UI definition (matches App class name)
├── screens/     # Optional: separate .py files per screen
├── images/      # Image assets
└── buildozer.spec # Config for packaging to Android/iOS

```

11. Packaging Your App (Brief Overview)

To turn a Kivy app into an installable Android/iOS app, the standard tool is **Buildozer**:

```
pip install buildozer
buildozer init      # creates buildozer.spec
buildozer -v android debug # builds a debug APK (Linux only)
```

Note: Buildozer for Android builds only runs reliably on Linux (or WSL). For iOS, Kivy uses a separate tool called kivy-ios.

12. Suggested Learning Path

1. Get comfortable with plain widgets and layouts in Python (Sections 3–5).
 2. Move UI definitions into .kv files once your layouts get more than a few widgets (Section 6).
 3. Learn Properties so you understand *why* KV bindings update automatically (Section 7).
 4. Build a small multi-screen app using ScreenManager (Section 9).
 5. Try packaging a simple app with Buildozer to see the mobile build process end-to-end.
-

13. Quick Reference – Common Imports

```
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.gridlayout import GridLayout
from kivy.uix.floatlayout import FloatLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
from kivy.uix.textinput import TextInput
from kivy.uix.image import Image
from kivy.uix.screenmanager import ScreenManager, Screen
from kivy.properties import StringProperty, NumericProperty, BooleanProperty, ListProperty
from kivy.clock import Clock # for scheduling repeated/delayed calls
```

14. Official Resources

- Documentation: <https://kivy.org/doc/stable/>
- GitHub: <https://github.com/kivy/kivy>
- KV language guide: <https://kivy.org/doc/stable/guide/lang.html>

This note covers the core mental model of Kivy: **widgets** are your building blocks, **layouts** arrange them, **KV language** describes them declaratively, and **Properties + events** make everything reactive. Once these four ideas click, everything else in Kivy (animations, custom widgets, ScreenManager transitions) builds naturally on top.